

数值算法 Homework 12

姓名: 雍崔扬

学号: 21307140051

Due: 08:00 May 12, 2025

Problem 1

Create a discontinuous function and smoothen it by convolving with Gaussian functions and Friedrichs mollifiers.

Make plots to visualize the results.

Note that Friedrichs mollifiers are functions of the form $\eta_\varepsilon(x) = \eta(x/\varepsilon)$ with $\varepsilon > 0$, where

$$\eta(x) = \begin{cases} \exp(\frac{1}{x^2-1}), & \text{if } |x| < 1 \\ 0, & \text{if } |x| \geq 1 \end{cases}$$

and the Gaussian mollifiers are functions of the form

$$\phi_\varepsilon(x) = \frac{1}{\varepsilon} \phi\left(\frac{x}{\varepsilon}\right) = \frac{1}{\sqrt{2\pi}\varepsilon} \exp\left(-\frac{x^2}{2\varepsilon^2}\right)$$
$$\text{where } \phi(x) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{x^2}{2}\right)$$

(optional) Apply the same technique to bivariable functions.

Solution:

设 $\{f[k]\}$ 和 $\{g[k]\}$ 是以 N 为周期的点列, 则其 (循环) 离散卷积定义为:

$$(f \star g)[k] := \sum_{j=0}^{N-1} f[j] \cdot g[k-j] \quad (k = 0, 1, \dots, N-1)$$

直接使用上式进行计算, 共需 N^2 次乘法和 $N(N-1)$ 次加法.

根据卷积定理 $\widetilde{(f \star g)}_{\text{DFT}}[j] = \tilde{f}_{\text{DFT}}[j] \cdot \tilde{g}_{\text{DFT}}[j]$ ($j = 0, 1, \dots, N-1$) 可知, 我们可以将离散卷积分解为以下三步来做:

- ① 计算 $\{f[k]\}$ 和 $\{g[k]\}$ 的离散 Fourier 变换: (必须使用 FFT)

$$\tilde{f}_{\text{DFT}}[j] = \sum_{k=0}^{N-1} f[k] \exp\left(-\frac{i2\pi jk}{N}\right)$$
$$\tilde{g}_{\text{DFT}}[j] = \sum_{k=0}^{N-1} g[k] \exp\left(-\frac{i2\pi jk}{N}\right)$$
$$\text{where } j = 0, 1, \dots, N-1$$

- ② 计算离散 Fourier 变换的乘积:

$$\widetilde{(f \star g)}_{\text{DFT}}[j] = \tilde{f}_{\text{DFT}}[j] \cdot \tilde{g}_{\text{DFT}}[j] \quad (j = 0, 1, \dots, N-1)$$

- ③ 计算离散 Fourier 逆变换: (必须使用 IFFT)

$$(f \star g)[k] = \frac{1}{N} \sum_{j=0}^{N-1} \widetilde{(f \star g)}_{\text{DFT}}[j] \exp\left(\frac{i2\pi jk}{N}\right)$$

where $k = 0, 1, \dots, N-1$

复用 Homework 11 Problem 3 给出的 FFT 实现:

```
function y = bitrevorder(x)
    % In-place bit-reversal permutation of vector x
    n = numel(x);
    r = log2(n);
    if floor(r)~=r
        error('bitrevorder: length must be power of 2');
    end
    y = zeros(size(x));
    for k = 0:n-1
        % reverse bits of k
        b = dec2bin(k, r);
        b_rev = b(end:-1:1);
        j = bin2dec(b_rev);
        y(j+1) = x(k+1);
    end
end

function x = fft_radix2_Coooley_Tukey(x)
    % Cooley-Tukey, decimation-in-time, non-recursive
    N = length(x);
    if mod(N,2)~=0 || log2(N)~=floor(log2(N))
        error('fft_radix2_dit: N must be 2^r');
    end
    x = bitrevorder(x);
    stages = log2(N);
    for s = 1:stages
        m = 2^s;
        half = m/2;
        w = exp(-2i*pi*(0:half-1)/m);
        for k = 1:m:N
            for j = 0:half-1
                t = w(j+1)*x(k+j+half);
                u = x(k+j);
                x(k+j) = u + t;
                x(k+j+half) = u - t;
            end
        end
    end
    x = x;
end
```

离散 Fourier 逆变换 (IDFT) 的快速算法可由快速 Fourier 变换导出:

$$f = \text{IFFT}(\tilde{f}_{\text{DFT}}) = \frac{1}{N} \cdot \overline{\text{FFT}\left(\overline{\tilde{f}_{\text{DFT}}}\right)}$$

```

% Inverse FFT using radix-2 FFT routine
function y = ifft_radix2(X)
    N = numel(X);
    % conjugate, forward FFT, conjugate, scale
    y = conj(fft_radix2_Coooley_Tukey(conj(X)))/N;
end

```

利用快速 Fourier 变换实现离散卷积:

```

% Convolution via zero-padded FFT
function h = fft_convolve(f, phi, dx)
    % pad to length 2N
    N0 = numel(f);
    F = fft_radix2_Coooley_Tukey([f; zeros(N0,1)]);
    P = fft_radix2_Coooley_Tukey([phi; zeros(N0,1)]);
    H = F .* P;
    h_full = ifft_radix2(H);
    % extract central part and scale by dx
    start_index = N0/2 + 1;
    end_index = start_index + N0 - 1;
    h = real(h_full(start_index:end_index)) * dx;
end

```

函数调用:

```

%--- Parameters ---
N = 2^12; % number of samples (power of two)
x = linspace(-1,1,N)'; % sample grid
dx = x(2)-x(1);

% Define discontinuous function f
f = zeros(N,1);
f(x<0) = x(x<0) + 1;
f(x>=0) = x(x>=0) - 1;

% Prepare figure for both subplots
figure;
set(gcf, 'Position', [100, 100, 1200, 500]); % Optional: make figure wider

%--- Subplot 1: Gaussian mollification ---
% Epsilon values to test
eps_vals = [0.07, 0.2, 0.5];

% Define distinct colors for each mollified curve
colors = lines(length(eps_vals)); % MATLAB's built-in distinct color palette

subplot(1,2,1);
plot(x, f, 'k--', 'Linewidth', 1.5); hold on;
for i = 1:length(eps_vals)
    eps = eps_vals(i);
    phi = gaussian_mollifier(x, eps);
    fs = fft_convolve(f, phi, dx);

```

```

    plot(x, fs, 'Linewidth', 1.5, 'color', colors(i,:));
end
title('Gaussian Mollification');
xlabel('x'); ylabel('f * \phi_\epsilon(x)');
legend_str = ["Original", arrayfun(@(e) sprintf('\epsilon = %.2f', e), eps_vals,
'UniformOutput', false)];
legend(legend_str, 'Location', 'best');
grid on;

%--- Subplot 2: Friedrichs mollification ---
% Epsilon values to test
eps_vals = [0.2, 0.5, 1.0];

% Define distinct colors for each mollified curve
colors = lines(length(eps_vals)); % MATLAB's built-in distinct color palette

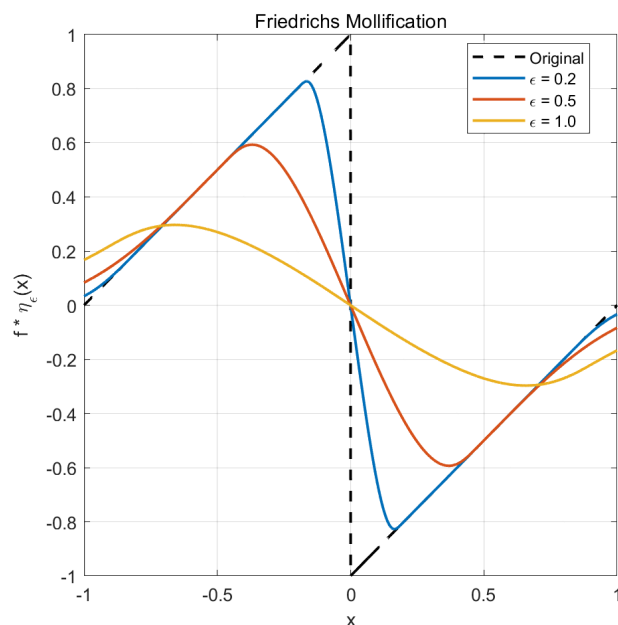
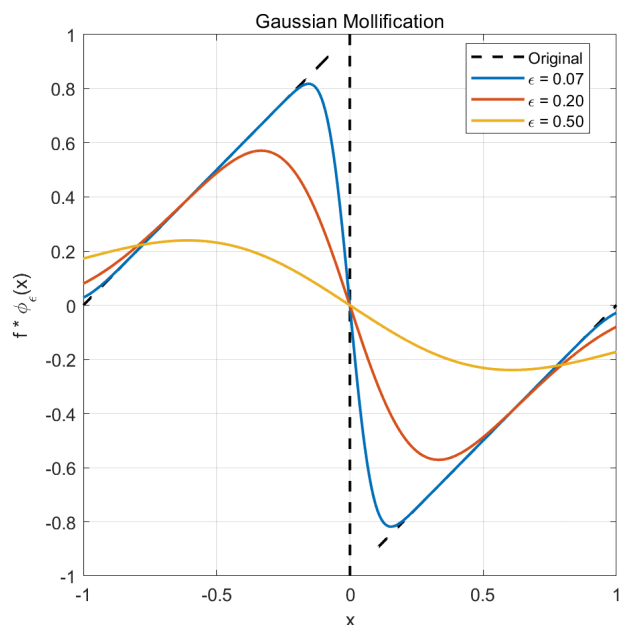
subplot(1,2,2);
plot(x, f, 'k--', 'Linewidth', 1.5); hold on;
for i = 1:length(eps_vals)
    eps = eps_vals(i);
    eta = friedrichs_mollifier(x, eps);
    fs = fft_convolve(f, eta, dx);
    plot(x, fs, 'Linewidth', 1.5, 'color', colors(i,:));
end
title('Friedrichs Mollification');
xlabel('x'); ylabel('f * \eta_\epsilon(x)');
legend_str = ["Original", arrayfun(@(e) sprintf('\epsilon = %.1f', e), eps_vals,
'UniformOutput', false)];
legend(legend_str, 'Location', 'best');
grid on;

% Mollifier definitions
function phi = gaussian_mollifier(x, eps)
    phi = exp(-x.^2/(2*eps^2)) / (sqrt(2*pi)*eps);
end

function eta = friedrichs_mollifier(x, eps)
    y = x/eps;
    eta = zeros(size(y));
    mask = abs(y) < 1;
    eta(mask) = exp(1./(y(mask).^2 - 1));
    % Normalize to ensure integral is 1
    C = trapz(x, eta);
    if C ~= 0
        eta = eta / C;
    end
end
end

```

运行结果:



Problem 2

The classical Runge-Kutta method (RK4) for solving the IVP

$$\begin{cases} u'(t) = f(t, u(t)) & (t > 0) \\ u(0) = u_0 \end{cases}$$

reads

$$u_{n+1} = u_n + \frac{h}{6}(s_1 + 2s_2 + 2s_3 + s_4)$$

where

$$\begin{aligned} s_1 &= f(t_n, u_n) \\ s_2 &= f\left(t_n + \frac{1}{2}h, u_n + \frac{1}{2}hs_1\right) \\ s_3 &= f\left(t_n + \frac{1}{2}h, u_n + \frac{1}{2}hs_2\right) \\ s_4 &= f(t_n + h, u_n + hs_3) \end{aligned}$$

Determine (experimentally) the order of global truncation error (GTE) for RK4.

(optional) Visualize the stable region of RK4.

- (我还以为要理论证明呢, 吓死了, [Prove that Runge Kutta Method \(RK4\) is of Order 4](#))

简单起见, 考虑 $u(t)$ 为一维的情况.

考虑把常微分方程的精确解 $u(t)$ 在 t_n 处展开:

$$u(t_{n+1}) = u(t_n) + hu'(t_n) + \frac{h^2}{2}u''(t_n) + \frac{h^3}{6}u^{(3)}(t_n) + \frac{h^4}{24}u^{(4)}(t_n) + O(h^5)$$

$$u'(t_n) = \frac{du}{dt} = f(t_n, u(t_n)) = f$$

$$u''(t_n) = \frac{df}{dt} = f'_t + f'_u f \equiv F$$

$$u^{(3)}(t_n) = \frac{d^2f}{dt^2} = f''_{tt} + 2f''_{tu}f + f''_{uu}f^2 + f'_u(f'_t + f'_u f) \equiv f'_u F + G$$

$$u^{(4)}(t_n) = \frac{d^3f}{dt^3} = f'''_{ttt} + 3f'''_{ttu}f + 3f'''_{tuu}f^2 + f'''_{uuu}f^3$$

+

where $F := f'_t + f'_u f$, $G := f''_{tt} + 2f''_{tu}f + f''_{uu}f^2$ and $H := f'''_{ttt} + 3f'''_{ttu}f + 3f'''_{tuu}f^2 + f'''_{uuu}f^3$

Solution:

考虑一阶常微分方程初值问题:

$$\begin{aligned} \frac{du(t)}{dt} &= f(t, u(t)) \\ u(t_0) &= u_0 \end{aligned}$$

四阶 Runge-Kutta 法 (经典 RK4) 的迭代格式为:

$$s_1 = f_n = f(t_n, u_n)$$

$$s_2 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_1\right)$$

$$s_3 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_2\right)$$

$$s_4 = f(t_n + h, u_n + hs_3)$$

$$u_{n+1} = u_n + \frac{h}{6}(s_1 + 2s_2 + 2s_3 + s_4)$$

c_1	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$	0	0	0	0	0
c_2	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$	1/2	1/2	0	0	0
c_3	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$	1/2	0	1/2	0	0
c_4	$a_{4,1}$	$a_{4,2}$	$a_{4,3}$	$a_{4,4}$	1	0	0	1	0
	b_1	b_2	b_3	b_4		1/6	1/3	1/3	1/6

理论上单步截断误差为 $O(h^5)$, 全局截断误差为 $O(h^4)$

收敛域为:

$$\left\{ z \in \mathbb{C} : \left| 1 + z + \frac{z^2}{2!} + \frac{z^3}{3!} + \frac{z^4}{4!} \leq 1 \right| \right\}$$

我们可以一阶线性时不变系统进行验证:

$$\frac{du(t)}{dt} = Au(t) + b(t)$$

$$u(t_0) = u_0$$

where $A \in \mathbb{C}^{n \times n}$ and $u(t), b(t) \in \mathbb{C}^n$

实验验证的 Matlab 代码:

```

% 函数定义
f = @(t, u) -u;
u_exact = @(t) exp(-t);
t_end = 1;

% 步长列表 (从 0.25 到 2^-7)
hs = 0.5 .^ (2:7);
errors = zeros(size(hs));

% 逐个步长计算误差
for k = 1:length(hs)
    h = hs(k);
    u_num = rk4(f, 0, 1.0, t_end, h);
    errors(k) = abs(u_exact(t_end) - u_num);
end

% 绘图: log-log 收敛图
figure;
loglog(hs, errors, 'o-', 'Linewidth', 2);
xlabel('Step size h');
ylabel('Global error at t=1');
title('RK4 Convergence (Global Truncation Error)');
grid on;

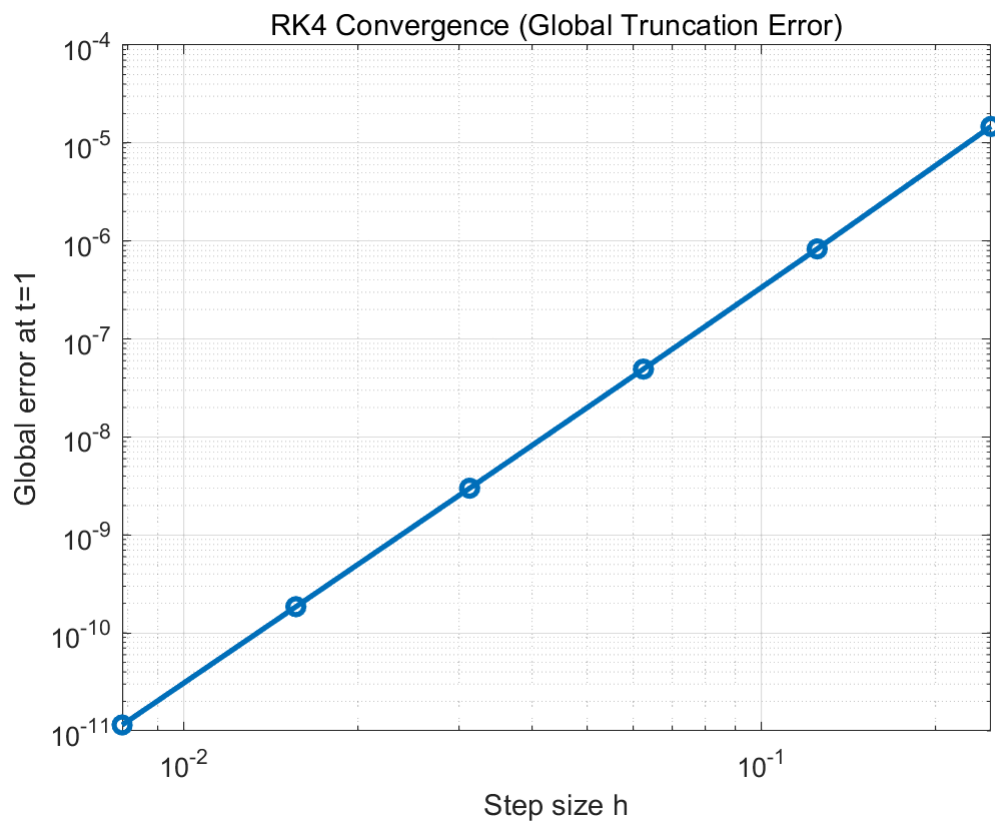
% 拟合斜率, 估算误差阶数
p = polyfit(log(hs), log(errors), 1);
fprintf('Estimated order of convergence: %.4f\n', p(1));

% RK4 实现
function u = rk4(f, t0, u0, t_end, h)
    N = round((t_end - t0) / h);
    u = u0;
    t = t0;
    for n = 1:N
        s1 = f(t, u);
        s2 = f(t + h/2, u + h/2 * s1);
        s3 = f(t + h/2, u + h/2 * s2);
        s4 = f(t + h, u + h * s3);
        u = u + h/6 * (s1 + 2*s2 + 2*s3 + s4);
        t = t + h;
    end
end

```

运行结果:

```
Estimated order of convergence: 4.0541
```



可视化收敛域:

```
% 可视化 RK4 收敛域（绿色）与外部区域（红色）
% -----
x = linspace(-4, 4, 800); % 实部范围
y = linspace(-4, 4, 800); % 虚部范围
[X, Y] = meshgrid(x, y);
Z = X + 1i*Y;

% 稳定性函数 R(z)
R = 1 + Z + Z.^2/2 + Z.^3/6 + Z.^4/24;
modR = abs(R);

% 创建 RGB 图像: 先初始化为红色（外部区域）
img = zeros([size(modR), 3]); % 高×宽×3 通道
img(:, :, 1) = 1; % R=1, G=0, B=0 => 红色

% 掩码: 内部区域 |R(z)| <= 1
inside = modR <= 1;
img(:, :, 1) = img(:, :, 1) .* ~inside; % 红色通道去除内部
img(:, :, 2) = inside; % 绿色通道填充内部

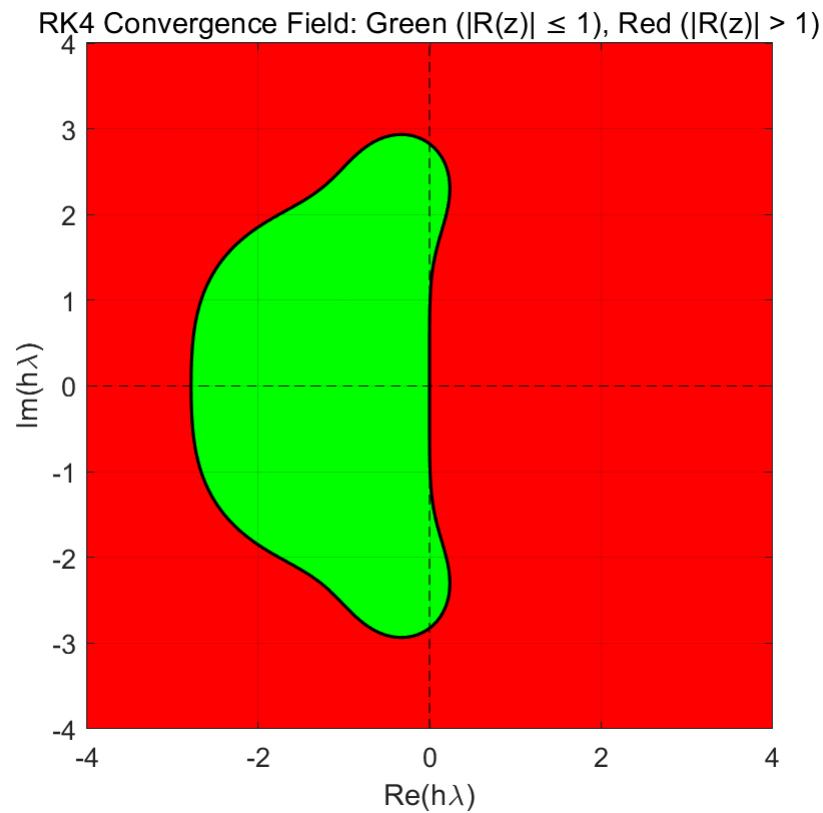
% 绘图
figure;
image(x, y, img);
axis xy;
axis equal;
xlabel('Re(h\lambda)');
ylabel('Im(h\lambda)');
title('RK4 Convergence Field: Green (|R(z)| ≤ 1), Red (|R(z)| > 1)');
```

```

grid on;

% 添加边界线 (可选)
hold on;
contour(X, Y, modR, [1 1], 'k', 'Linewidth', 1.2);
plot([0 0], ylim, 'k--'); % 虚轴
plot(xlim, [0 0], 'k--'); % 实轴

```



Problem 3

Bacteria growing in a batch reactor utilize a soluble food source (substrate) as depicted in following figure.

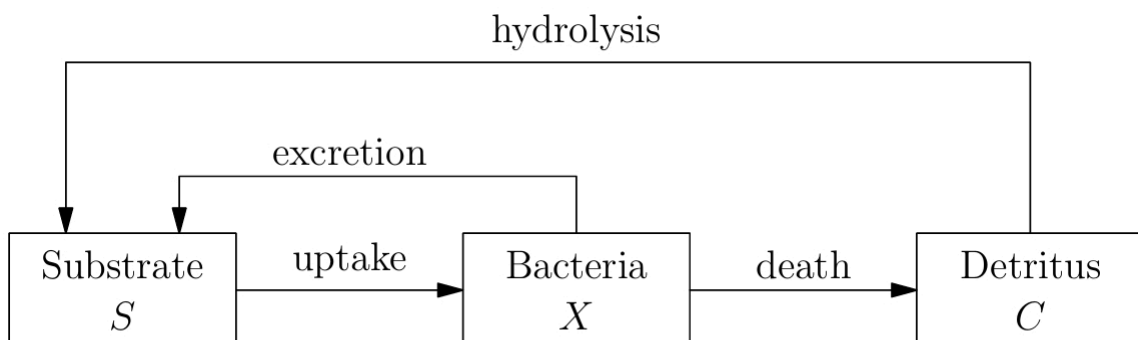


Figure 1: A batch reactor for bacteria growth.

The uptake of the substrate is represented by a logistic model with Michaelis–Menten limitation.

Death of the bacteria produces detritus (碎屑)

which is subsequently converted to the substrate (底物) by hydrolysis (水解).

In addition, the bacteria also excrete (排泄) some substrate directly.

Death, hydrolysis and excretion are all simulated as first-order reactions.

Mass balances can be written as

$$\begin{cases} X' = \mu_{\max} \left(1 - \frac{X}{K}\right) \left(\frac{S}{K_S + S}\right) X - k_d X - k_e X \\ C' = k_d X - k_h C \\ S' = k_e X + k_h C - \mu_{\max} \left(1 - \frac{X}{K}\right) \left(\frac{S}{K_S + S}\right) X \end{cases}$$

where X , C , and S are the concentrations [$\text{mg} \cdot \text{L}^{-1}$] of bacteria, detritus, and substrate, respectively;

$\mu_{\max}$ is the maximum growth rate [d^{-1}];

K is the logistic carrying capacity [$\text{mg} \cdot \text{L}^{-1}$];

K_S is the Michaelis–Menten half-saturation constant [$\text{mg} \cdot \text{L}^{-1}$];

k_d , k_e , and k_h , respectively, are the death rate [d^{-1}], the excretion rate [d^{-1}], and the hydrolysis rate [d^{-1}].

Simulate the concentrations from $t = 0$ to 100d, given the initial conditions

$$\begin{aligned} X(0) &= 1 \text{ mg} \cdot \text{L}^{-1} \\ S(0) &= 100 \text{ mg} \cdot \text{L}^{-1} \\ C(0) &= 0 \text{ mg} \cdot \text{L}^{-1} \end{aligned}$$

Employ the following parameters in your calculation:

$$\begin{aligned} \mu_{\max} &= 10 \text{ d}^{-1} \\ K &= 10 \text{ mg} \cdot \text{L}^{-1} \\ K_S &= 10 \text{ mg} \cdot \text{L}^{-1} \\ k_d = k_e = k_h &= 0.1 \text{ d}^{-1} \end{aligned}$$

Find stationary concentrations and visualize your solution.

Solution:

考虑一阶常微分方程初值问题:

$$\begin{aligned} \frac{du(t)}{dt} &= f(t, u(t)) \\ u(t_0) &= u_0 \end{aligned}$$

四阶 Runge-Kutta 法 (经典 RK4) 的迭代格式为:

$$s_1 = f_n = f(t_n, u_n)$$

$$s_2 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_1\right)$$

$$s_3 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_2\right)$$

$$s_4 = f(t_n + h, u_n + hs_3)$$

$$u_{n+1} = u_n + \frac{h}{6}(s_1 + 2s_2 + 2s_3 + s_4)$$

c_1	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$	0	0	0	0	0
c_2	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$	$1/2$	$1/2$	0	0	0
c_3	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$	$= 1/2$	0	$1/2$	0	0
c_4	$a_{4,1}$	$a_{4,2}$	$a_{4,3}$	$a_{4,4}$	1	0	0	1	0
	b_1	b_2	b_3	b_4		$1/6$	$1/3$	$1/3$	$1/6$

Matlab 实现:

```
% Parameters
params.mu_max = 10;
params.K = 10;
params.Ks = 10;
params.kd = 0.1;
params.ke = 0.1;
params.kh = 0.1;

% Initial conditions
y0 = [1; 0; 100]; % x, c, s

% Time settings
t_start = 0;
t_end = 100;
h = 0.1;
num_steps = (t_end - t_start) / h;
t = linspace(t_start, t_end, num_steps + 1);

% Initialize solution array
Y = zeros(3, num_steps + 1);
Y(:,1) = y0;

% RK4 loop
for n = 1:num_steps
    current_t = t(n);
    current_y = Y(:,n);

    k1 = derivatives(current_t, current_y, params);
    k2 = derivatives(current_t + h/2, current_y + h/2*k1, params);
    k3 = derivatives(current_t + h/2, current_y + h/2*k2, params);
    k4 = derivatives(current_t + h, current_y + h*k3, params);

    Y(:,n+1) = current_y + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
end
```

```

% Extract variables
X = Y(1,:);
C = Y(2,:);
S = Y(3,:);

% Plot
figure;
plot(t, X, 'r', t, C, 'g', t, S, 'b', 'Linewidth', 1.5);
legend('Bacteria (X)', 'Detritus (C)', 'Substrate (S)');
xlabel('Time (days)');
ylabel('Concentration (mg/L)');
title('Dynamics of Bacteria, Detritus, and Substrate');
grid on;

% Display stationary concentrations
fprintf('Stationary concentrations at t=100 days:\n');
fprintf('X: %.4f mg/L\n', X(end));
fprintf('C: %.4f mg/L\n', C(end));
fprintf('S: %.4f mg/L\n', S(end));

% Derivative function
function dydt = derivatives(t, y, params)
    X = y(1);
    C = y(2);
    S = y(3);

    mu_max = params.mu_max;
    K = params.K;
    Ks = params.Ks;
    kd = params.kd;
    ke = params.ke;
    kh = params.kh;

    growth_term = mu_max * (1 - X/K) * (S / (Ks + S)) * X;

    dxdt = growth_term - kd*X - ke*X;
    dCdt = kd*X - kh*C;
    dSdt = ke*X + kh*C - growth_term;

    dydt = [dxdt; dCdt; dSdt];
end

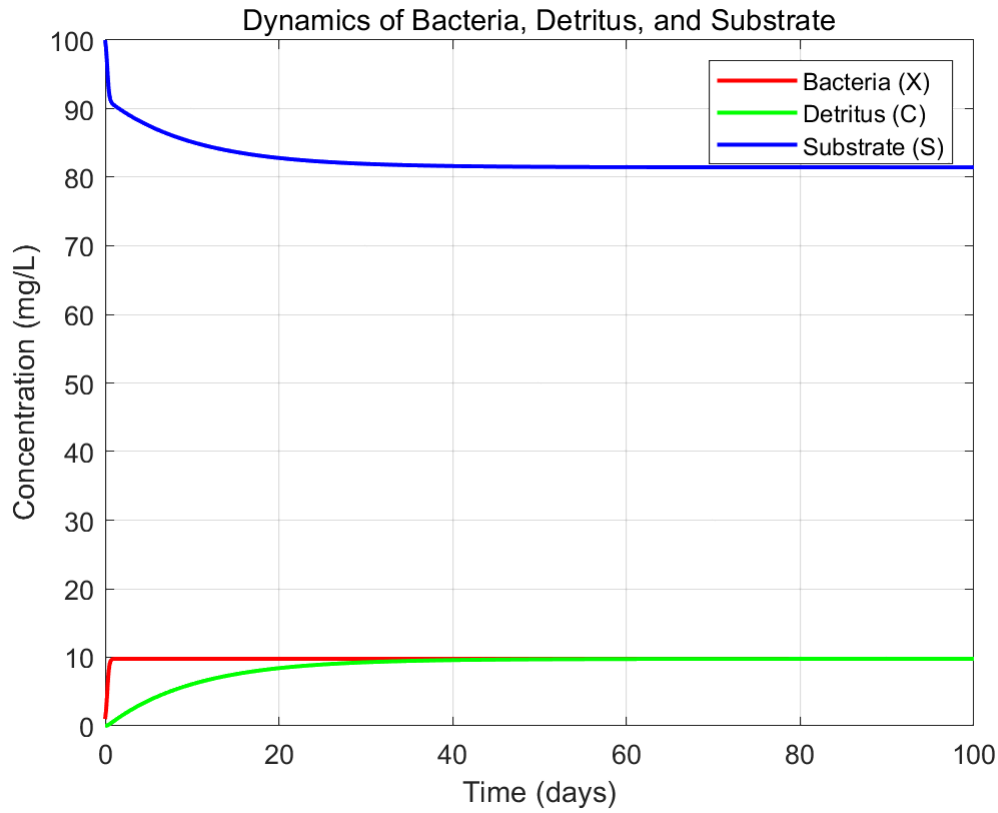
```

运行结果:

```

Stationary concentrations at t=100 days:
X: 9.7754 mg/L
C: 9.7750 mg/L
S: 81.4496 mg/L

```



Problem 4

A spring-mass system as shown in figure below can be modeled by the following second order ODE,

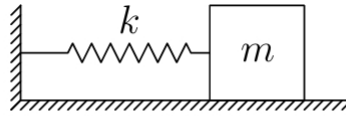


Figure 2: A simple spring-mass system.

under certain simplifying assumptions (e.g., small displacement, spring has no mass, no friction, etc.):

$$\begin{aligned} mx''(t) &= -kx(t) \\ x(0) &= x_0 \\ x'(0) &= v_0 \end{aligned}$$

where m is the weight of the mass and k is the spring constant.

- (a) Find the exact solution $x(t)$.
- (b) Transform the second order ODE to a first order one by introducing $u(t) = [x(t), x'(t)]^T$.
- (c) Solve the IVP in part (b) by Euler method, backward Euler method, trapezoidal method and classical Runge-Kutta method, for a long period. You may assume $m = k = 1$, and use a step size $h = 0.1$. Visualize your solutions using phase diagrams (i.e., plot the solutions in the u -plane). What can you say about long-term behavior of these methods?

Part (a)

Find the exact solution $x(t)$.

Solution:

注意到 $mx''(t) = -kx(t)$ 的特征方程为 $mr^2 = -k$, 其根为 $r = \pm i\sqrt{k/m}$
因此解具有如下形式:

$$x(t) = C_1 \cos(\sqrt{\frac{k}{m}}t) + C_2 \sin(\sqrt{\frac{k}{m}}t)$$

其导数为:

$$x'(t) = -\sqrt{\frac{k}{m}}C_1 \sin(\sqrt{\frac{k}{m}}t) + \sqrt{\frac{k}{m}}C_2 \cos(\sqrt{\frac{k}{m}}t)$$

代入 $x(0) = x_0$ 和 $x'(0) = v_0$ 可得:

$$\begin{aligned} x(0) &= C_1 \cos(0) + C_2 \sin(0) \\ &= C_1 \\ &= x_0 \\ \hline x'(0) &= -\sqrt{\frac{k}{m}}C_1 \sin(0) + \sqrt{\frac{k}{m}}C_2 \cos(0) \\ &= \sqrt{\frac{k}{m}}C_2 \\ &= v_0 \end{aligned}$$

可得:

$$\begin{cases} C_1 = x_0 \\ C_2 = \sqrt{\frac{m}{k}} \cdot v_0 \end{cases}$$

因此问题的解析解为:

$$x(t) = x_0 \cos(\sqrt{\frac{k}{m}}t) + \sqrt{\frac{m}{k}} \cdot v_0 \cdot \sin(\sqrt{\frac{k}{m}}t)$$

Part (b)

Transform the second order ODE to a first order one by introducing $u(t) = [x(t), x'(t)]^T$.

Solution:

记 $u_1(t) := x(t)$ 和 $u_2(t) := x'(t)$, 则我们有:

$$\begin{cases} u_1'(t) = x'(t) = u_2(t) \\ u_2'(t) = x''(t) = -\frac{k}{m}x(t) = -\frac{k}{m}u_1(t) \end{cases}$$

于是我们有:

$$u'(t) = \begin{bmatrix} u_1'(t) \\ u_2'(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & 0 \end{bmatrix} \begin{bmatrix} u_1(t) \\ u_2(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & 0 \end{bmatrix} u(t)$$

对应的一阶常微分方程初值问题为:

$$u'(t) = Au(t)$$

$$u(0) = \begin{bmatrix} x_0 \\ v_0 \end{bmatrix}$$

where $A = \begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & 0 \end{bmatrix}$

Part (c)

Solve the IVP in part (b) by Euler method, backward Euler method, trapezoidal method and classical Runge-Kutta method, for a long period.

You may assume $m = k = 1$, and use a step size $h = 0.1$.

Visualize your solutions using phase diagrams (i.e., plot the solutions in the u-plane).

What can you say about long-term behavior of these methods?

Solution:

- 显式 Euler 法:

$$u_{n+1} = u_n + hf_n$$

$$\Updownarrow$$

$$s_1 = f_n = f(t_n, u_n)$$

$$u_{n+1} = u_n + hs_1$$

$$\begin{array}{c|c} c_1 & a_{1,1} \\ \hline & b_1 \end{array} = \begin{array}{c|c} 0 & 0 \\ \hline & 1 \end{array}$$

- 隐式 Euler 法:

$$u_{n+1} = u_n + hf_{n+1}$$

$$\Updownarrow$$

$$s_1 = f_{n+1} = f(t_{n+1}, u_{n+1}) = f(t_n + h, u_n + hs_1)$$

$$u_{n+1} = u_n + hs_1$$

$$\begin{array}{c|c} c_1 & a_{1,1} \\ \hline & b_1 \end{array} = \begin{array}{c|c} 1 & 1 \\ \hline & 1 \end{array}$$

- 显式梯形法:

$$u_{n+1} = u_n + \frac{h}{2}(f_n + f(t_{n+1}, u_n + hf_n))$$

$$\Updownarrow$$

$$s_1 = f_n = f(t_n, u_n)$$

$$s_2 = f(t_{n+1}, u_n + hf_n) = f(t_{n+1}, u_n + hs_1)$$

$$u_{n+1} = u_n + \frac{h}{2}s_1 + \frac{h}{2}s_2$$

$$\begin{array}{c|ccc} c_1 & a_{1,1} & a_{1,2} & 0 \\ \hline c_2 & a_{2,1} & a_{2,2} & 1 \\ \hline & b_1 & b_2 & \end{array} = \begin{array}{c|cc} 0 & 0 & 0 \\ \hline 1 & 1 & 0 \\ \hline & 1/2 & 1/2 \end{array}$$

- 隐式梯形法:

$$u_{n+1} = u_n + \frac{h}{2}(f_n + f_{n+1})$$

$$\Updownarrow$$

$$s_1 = f_n = f(t_n, u_n)$$

$$s_2 = f_{n+1} = f(t_{n+1}, u_{n+1}) = f\left(t_{n+1}, u_n + \frac{h}{2}s_1 + \frac{h}{2}s_2\right)$$

$$u_{n+1} = u_n + \frac{h}{2}s_1 + \frac{h}{2}s_2$$

c_1	$a_{1,1}$	$a_{1,2}$	0	0	0
c_2	$a_{2,1}$	$a_{2,2}$	$= 1$	$1/2$	$1/2$
	b_1	b_2		$1/2$	$1/2$

• 三阶 Runge-Kutta 法:

$$s_1 = f_n = f(t_n, u_n)$$

$$s_2 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_1\right)$$

$$s_3 = f(t_n + h, u_n - hs_1 + 2hs_2)$$

$$u_{n+1} = u_n + \frac{h}{6}(s_1 + 4s_2 + s_3)$$

c_1	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	0	0	0	0
c_2	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$= 1/2$	$1/2$	0	0
c_3	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	1	-1	2	0
	b_1	b_2	b_3		$1/6$	$2/3$	$1/6$

• 四阶 Runge-Kutta 法: (经典 RK4)

$$s_1 = f_n = f(t_n, u_n)$$

$$s_2 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_1\right)$$

$$s_3 = f\left(t_n + \frac{h}{2}, u_n + \frac{h}{2}s_2\right)$$

$$s_4 = f(t_n + h, u_n + hs_3)$$

$$u_{n+1} = u_n + \frac{h}{6}(s_1 + 2s_2 + 2s_3 + s_4)$$

c_1	$a_{1,1}$	$a_{1,2}$	$a_{1,3}$	$a_{1,4}$	0	0	0	0	0
c_2	$a_{2,1}$	$a_{2,2}$	$a_{2,3}$	$a_{2,4}$	$1/2$	$1/2$	0	0	0
c_3	$a_{3,1}$	$a_{3,2}$	$a_{3,3}$	$a_{3,4}$	$= 1/2$	0	$1/2$	0	0
c_4	$a_{4,1}$	$a_{4,2}$	$a_{4,3}$	$a_{4,4}$	1	0	0	1	0
	b_1	b_2	b_3	b_4		$1/6$	$1/3$	$1/3$	$1/6$

Matlab 实现:

```
% Parameters for the harmonic oscillator
m = 1; k = 1;
A = [0 1; -k/m 0];

h = 0.1;           % time step
T = 1000;          % total simulation time
```

```

Nsteps = round(T/h);

% Initial condition [x; v]
u0 = [1; 0];

% Preallocate solution arrays
u_euler      = zeros(2, Nsteps+1);
u_beuler     = zeros(2, Nsteps+1);
u_trap_exp   = zeros(2, Nsteps+1);
u_trap_impl  = zeros(2, Nsteps+1);
u_rk3        = zeros(2, Nsteps+1);
u_rk4        = zeros(2, Nsteps+1);

% Set initial states
u_euler(:,1) = u0;
u_beuler(:,1) = u0;
u_trap_exp(:,1) = u0;
u_trap_impl(:,1) = u0;
u_rk3(:,1) = u0;
u_rk4(:,1) = u0;

I = eye(2); % identity for implicit solves

% Time-stepping loop
for n = 1:Nsteps
    un_e = u_euler(:,n);
    un_b = u_beuler(:,n);
    un_te = u_trap_exp(:,n);
    un_ti = u_trap_impl(:,n);
    un_3 = u_rk3(:,n);
    un_4 = u_rk4(:,n);

    % 1) Explicit Euler
    u_euler(:,n+1) = un_e + h*(A*un_e);

    % 2) Implicit (Backward) Euler
    u_beuler(:,n+1) = (I - h*A) \ un_b;

    % 3) Explicit Trapezoidal (Heun's method)
    s1 = A * un_te;
    s2 = A * (un_te + h * s1);
    u_trap_exp(:,n+1) = un_te + (h/2)*(s1 + s2);

    % 4) Implicit Trapezoidal (Crank-Nicolson)
    u_trap_impl(:,n+1) = (I - (h/2)*A) \ ((I + (h/2)*A)*un_ti);

    % 5) Third-order Runge-Kutta (RK3)
    k1 = A * un_3;
    k2 = A * (un_3 + (h/2)*k1);
    k3 = A * (un_3 - h*k1 + 2*h*k2);
    u_rk3(:,n+1) = un_3 + (h/6)*(k1 + 4*k2 + k3);

    % 6) Classical RK4

```

```

    K1 = A * un_4;
    K2 = A * (un_4 + (h/2)*K1);
    K3 = A * (un_4 + (h/2)*K2);
    K4 = A * (un_4 + h*K3);
    u_rk4(:,n+1) = un_4 + (h/6)*(K1 + 2*K2 + 2*K3 + K4);
end

% Plot phase diagrams
figure('Position',[100 100 1200 800]);

subplot(2,3,1);
plot(u_euler(1,:), u_euler(2,:));
title('Explicit Euler');
xlabel('x'); ylabel('v'); axis equal;

subplot(2,3,2);
plot(u_beuler(1,:), u_beuler(2,:));
title('Implicit Euler');
xlabel('x'); ylabel('v'); axis equal;

subplot(2,3,3);
plot(u_trap_exp(1,:), u_trap_exp(2,:));
title('Explicit Trapezoidal');
xlabel('x'); ylabel('v'); axis equal;

subplot(2,3,4);
plot(u_trap_impl(1,:), u_trap_impl(2,:));
title('Implicit Trapezoidal');
xlabel('x'); ylabel('v'); axis equal;

subplot(2,3,5);
plot(u_rk3(1,:), u_rk3(2,:));
title('Explicit RK3');
xlabel('x'); ylabel('v'); axis equal;

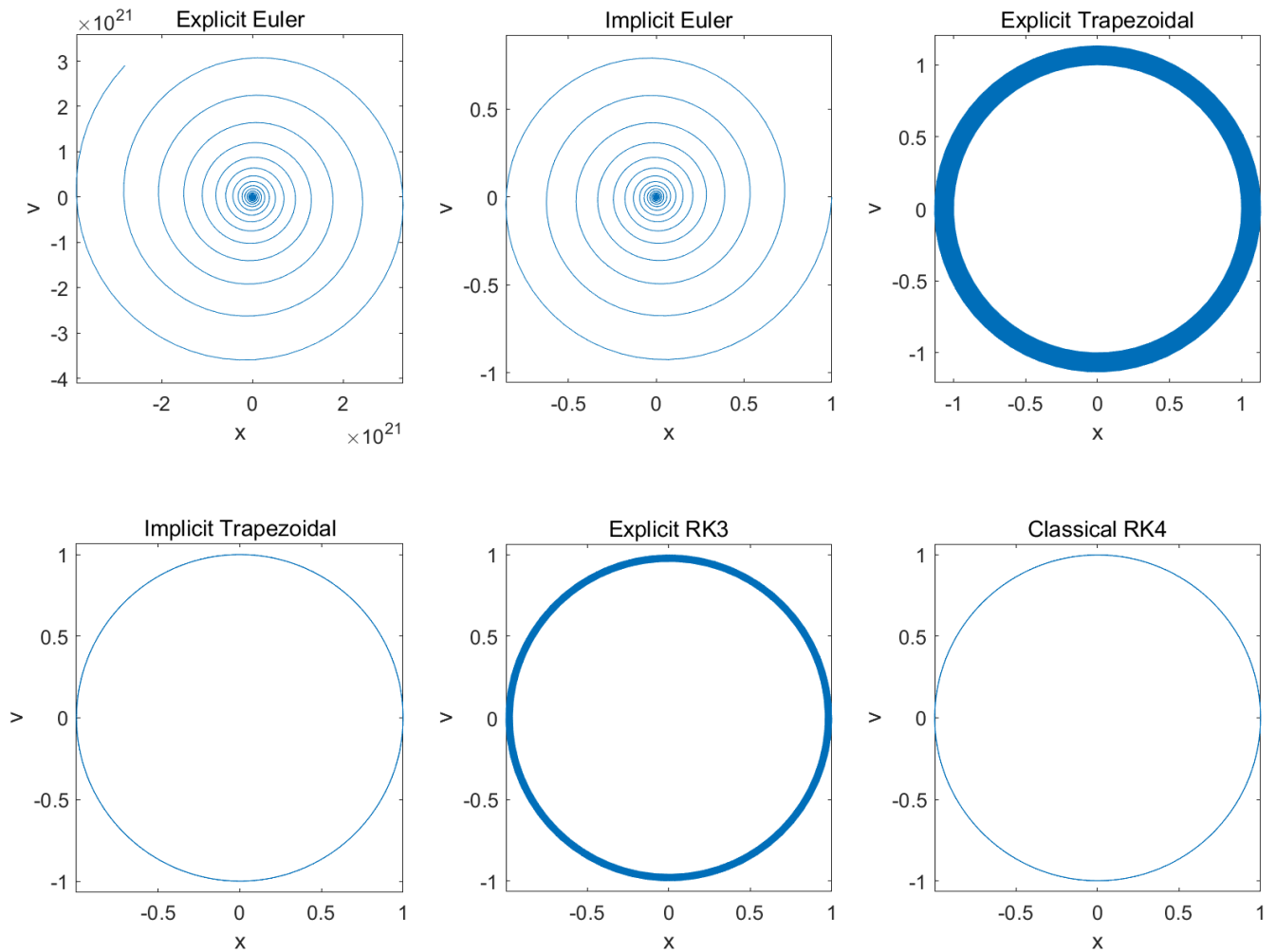
subplot(2,3,6);
plot(u_rk4(1,:), u_rk4(2,:));
title('Classical RK4');
xlabel('x'); ylabel('v'); axis equal;

% Overall title
sgtitle('Phase Portraits for Six Integration Schemes');

```

运行结果:

Phase Portraits for Six Integration Schemes



对于简谐振子系统来说，上述六种方法有如下性质：

- 显式 Euler 法呈明显的发散螺旋。
当 $k = m = 1$ 时 A 的特征值为 $\lambda_{1,2} = \pm i$ ，不满足收敛条件 $|1 + h\lambda| < 1$
- 隐式 Euler 法稳定但耗散
- 显式梯形法稳定但耗散，不过耗散程度弱于隐式 Euler 法
- 隐式梯形法稳定且不耗散，适合长期模拟
- 三阶 Runge-Kutta 法稳定但耗散，不过耗散程度弱于显式梯形法
- 四阶 Runge-Kutta 法稳定且不耗散，适合长期模拟

总之我们有如下结论：

- 若关心长期能量守恒，首选隐式梯形法 (Crank-Nicolson)
- 若只需短期高精度模拟，四阶 Runge-Kutta 法是好的选择。
- 对于振荡系统，应尽量避免使用 Euler 类方法，无论显式或隐式。

Problem 5 (optional)

Prove that

$$\delta(x) = \frac{1}{\pi} \lim_{t \rightarrow 0+} \frac{t^2}{x^2 + t^2}$$

It suffices to show

$$\lim_{t \rightarrow 0+} \int_0^1 \frac{tf(x)}{x^2 + t^2} dx = \frac{\pi}{2} f(0)$$

for any continuous function $f(x)$.

Solution:

任取 $\varepsilon > 0$, 存在 $\eta \in (0, 1)$ 使得当 $x \in [0, \eta]$ 时恒有 $|f(x) - f(0)| < \varepsilon/\pi$ 成立.

于是对于任意 $t > 0$, 我们都有:

$$\begin{aligned} \left| \int_0^\eta \frac{tf(x)}{x^2 + t^2} dx - \int_0^\eta \frac{tf(0)}{x^2 + t^2} dx \right| &= \left| \int_0^\eta \frac{t(f(x) - f(0))}{x^2 + t^2} dx \right| \\ &\leq \int_0^\eta \frac{t|f(x) - f(0)|}{x^2 + t^2} dx \\ &\leq \frac{\varepsilon}{\pi} \int_0^\eta \frac{t}{x^2 + t^2} dx \\ &= \frac{\varepsilon}{\pi} \arctan\left(\frac{x}{t}\right) \Big|_0^\eta \\ &= \frac{\varepsilon}{\pi} \arctan\left(\frac{\eta}{t}\right) \\ &< \frac{\varepsilon}{\pi} \cdot \frac{\pi}{2} \\ &= \frac{\varepsilon}{2} \end{aligned}$$

设 $|f(x)|$ 在 $[0, 1]$ 上的最大值为 M

取 $\delta = \eta^2 \varepsilon / [2M(1 - \eta)]$

当 $t \in (0, \delta)$ 时, 我们有:

$$\begin{aligned} \left| \int_\eta^1 \frac{tf(x)}{x^2 + t^2} dx \right| &\leq \int_\eta^1 \frac{t|f(x)|}{x^2 + t^2} dx \\ &\leq M \cdot \int_\eta^1 \frac{t}{x^2 + t^2} dx \\ &< M \cdot \frac{t}{\eta^2} \cdot (1 - \eta) \\ &< M \cdot \frac{\eta^2 \varepsilon / [2M(1 - \eta)]}{\eta^2} \cdot (1 - \eta) \\ &= \frac{\varepsilon}{2} \end{aligned}$$

于是对于任意 $t \in (0, \delta)$, 我们都有:

$$\begin{aligned}
\left| \int_0^1 \frac{tf(x)}{x^2+t^2} dx - f(0) \arctan\left(\frac{\eta}{t}\right) \right| &= \left| \int_0^1 \frac{tf(x)}{x^2+t^2} dx - \int_0^\eta \frac{tf(0)}{x^2+t^2} dx \right| \\
&= \left| \int_\eta^1 \frac{tf(x)}{x^2+t^2} dx + \int_0^\eta \frac{tf(x)}{x^2+t^2} dx - \int_0^\eta \frac{tf(0)}{x^2+t^2} dx \right| \\
&\leq \left| \int_\eta^1 \frac{tf(x)}{x^2+t^2} dx \right| + \left| \int_0^\eta \frac{tf(x)}{x^2+t^2} dx - \int_0^\eta \frac{tf(0)}{x^2+t^2} dx \right| \\
&< \frac{\varepsilon}{2} + \frac{\varepsilon}{2} \\
&= \varepsilon
\end{aligned}$$

令 $t \rightarrow 0_+$ 可得:

$$\begin{aligned}
\left| \lim_{t \rightarrow 0_+} \int_0^1 \frac{tf(x)}{x^2+t^2} dx - \frac{\pi}{2} f(0) \right| &= \left| \lim_{t \rightarrow 0_+} \int_0^1 \frac{tf(x)}{x^2+t^2} dx - f(0) \cdot \lim_{t \rightarrow 0_+} \arctan\left(\frac{\eta}{t}\right) \right| \\
&\leq \lim_{t \rightarrow 0_+} \left| \int_0^1 \frac{tf(x)}{x^2+t^2} dx - f(0) \arctan\left(\frac{\eta}{t}\right) \right| \\
&< \varepsilon
\end{aligned}$$

因此我们有:

$$\lim_{t \rightarrow 0_+} \int_0^1 \frac{tf(x)}{x^2+t^2} dx = \frac{\pi}{2} f(0)$$

Problem 6 (optional)

Given a piece of monophonic music (e.g., flute/oboe/clarinet/etc. solo),

Fourier transform can be used to identify the fundamental tones of the music.

Write a program to play the music and show the fundamental tones simultaneously.

Problem 7 (optional)

Verify the solution of the brachistochrone problem by experiments.

You may choose a few curves and compare them with the true solution (a segment of a cycloid).